Nome e Cognome: _____ **Matricola:** _____

Istruzioni: Il tempo a disposizione per completare la prova è di **45 minuti**. Durante la prova, non è consentito consultare materiale didattico, né interagire con dispositivi elettronici o colleghi. Nel caso di domande a risposta multipla con più risposte corrette, ciascuna risposta corretta indicata vale una corrispondente frazione del punteggio complessivo, mentre le risposte errate comportano una pari penalizzazione. In nessun caso, il punteggio complessivo relativo a una domanda sarà negativo.

Es. **1.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
1  const voti = [18, 24, 30, 27, 21];
2  const r = voti
3    .filter(v => v >= 24)
4    .map(v => v + 1)
5    .reduce((somma, v) => somma + v, 0);
6  console.log(r, voti.length);
```

- A `81 3.`
- B `125 5.`
- **C** `84 5.`
- D `84 3.`
- E `81 5.`
- F `[ 25, 31, 28 ] 5.`

Risposta: C. `filter` tiene 24, 30 e 27; `map` li trasforma in 25, 31 e 28; `reduce` li somma a partire da 0: 84. Questi metodi restituiscono nuovi valori senza modificare l'array originale, che resta di 5 elementi. *(Dispensa: cap. 6, «Iterazione»)*

Es. **2.** **(2 punti)** Qual è l'output in console dopo il caricamento completo del documento HTML riportato di seguito?

```
1  <!DOCTYPE html>
2  <html>
3    <head><title>DOM</title></head>
4    <body>
5      <script> console.log(document.body.querySelector("h1")?.textContent) </script>
6      <h1>Hello</h1>
7    </body>
8  </html>
```

- A Viene sollevato un `TypeError`.
- B `Hello`.
- C `<h1>Hello</h1>`.
- **D** `undefined`.

Risposta: D. Lo script viene eseguito appena il parser lo incontra, quando l'`<h1>` che lo segue non è ancora nel DOM: `querySelector` restituisce `null`. L'operatore `?.` evita l'errore e dà `undefined` (senza `?.` si avrebbe un `TypeError`). *(Dispensa: cap. 7, «Ordine di esecuzione degli script») — Fonte: facsimile mostrato in aula.*

Es. **3.** **(2 punti)** Qual è l'output in console dopo il caricamento completo del documento HTML riportato di seguito?

```
1  <script>
2    localStorage.setItem("visite", 1);
3    localStorage.visite = Number(localStorage.getItem("visite")) + 1;
4    const v = localStorage.getItem("visite");
5    console.log(typeof v, v + 1, localStorage.getItem("utente"));
6  </script>
```

- A `number 3 null.`
- B `number 3 undefined.`
- C `string 21 undefined.`
- D `number 2 null.`
- **E** `string 21 null.`
- F `string 11 null.`

Risposta: E. `localStorage` memorizza **solo stringhe**: il numero viene convertito in stringa al salvataggio, e `getItem` restituisce `"2"`, per cui `v + 1` concatena. Le proprietà dell'oggetto sono un'alternativa a `getItem` e `setItem`. Per una chiave assente `getItem` restituisce `null`. *(Dispensa: cap. 8, «localStorage»)*

SOLUZIONI

Es. **4.** **(2 punti)** Quali delle seguenti affermazioni sul DNS sono **vere**?

- [A] Tutte le richieste di risoluzione arrivano a un unico server centrale.
- [B] Le risposte del DNS non vengono mai memorizzate, per essere sempre aggiornate.
- [C] Il DNS assegna gli indirizzi IP agli host quando si collegano alla rete.
- [**D**] Il browser di norma interroga un *resolver*, che scende nell'albero per suo conto.
- [E] Un nome si legge da sinistra a destra, dalla radice verso l'host.
- [**F**] I nomi sono organizzati in un albero: dalla radice ai TLD come `.it`, poi ai domini di secondo livello come `unina.it`.

Risposte: D, F. Il DNS è una rubrica distribuita e gerarchica: la radice sa chi gestisce ciascun TLD, il server del TLD sa chi gestisce ciascun dominio, il server autoritativo conosce gli indirizzi. Le risposte restano in cache per la durata del TTL. Gli indirizzi agli host li assegna DHCP. Un nome si legge da destra a sinistra risalendo verso la radice. *(Dispensa: cap. 9, «Dalla URL all'indirizzo: il DNS»)*

Es. **5.** **(2 punti)** Per cancellare un elemento, un'applicazione mostra accanto a ogni voce un link `<a href="/delete?id=3">`. Perché è una cattiva idea?

- [A] Perché l'identificativo dovrebbe essere la posizione dell'elemento nell'array.
- [B] Perché una `GET` non può ricevere come risposta una pagina HTML.
- [C] Perché il server non può leggere i parametri di una `GET`.
- [**D**] Perché una `GET` dovrebbe essere sicura: browser che precaricano i link, crawler o estensioni potrebbero cancellare elementi senza che nessuno l'abbia chiesto.
- [E] Perché i link non possono contenere parametri nella query string.
- [F] Non è una cattiva idea: le `GET` sono il metodo giusto per cancellare.

Risposta: D. Le operazioni che modificano lo stato del server vanno affidate a metodi non sicuri: qui un form con metodo `POST` e l'identificativo in un campo nascosto (o, in un'API, una `DELETE`). *(Dispensa: cap. 12, «Estendere l'applicazione: cancellare un elemento»)*

Es. **6.** **(2 punti)** Un'applicazione conserva le sessioni in una `Map` in memoria. Quali problemi può avere in produzione?

- [A] I dati della sessione diventano visibili al client.
- [**B**] Al riavvio del server tutte le sessioni spariscono, e gli utenti devono rifare il login.
- [**C**] Con più macchine dietro un bilanciatore di carico, una richiesta servita da una macchina non trova la sessione creata su un'altra.
- [D] Gli utenti possono modificare i dati della sessione dai DevTools.
- [E] Una `Map` non può contenere più di 20 sessioni.
- [F] Le sessioni scadono dopo ogni richiesta.

Risposte: B, C. La memoria del processo non sopravvive ai riavvii e non è condivisa fra più macchine. In produzione le sessioni si conservano in un archivio condiviso (Redis, un database), oppure si usano token autoconsistenti e firmati. *(Dispensa: cap. 13, «Usare le sessioni»)*

Es. **7.** **(2 punti)** Che cosa si intende con *inversione del controllo* (la «legge di Hollywood»), caratteristica distintiva dei framework?

- [A] Che le richieste HTTP vengono elaborate in ordine inverso.
- [B] Che il client controlla il server, invece del contrario.
- [C] Che il framework impone di scrivere il codice dalla fine all'inizio.
- [D] Che il database controlla la logica dell'applicazione.
- [**E**] Che è il framework a controllare il flusso di esecuzione e a invocare il codice scritto dallo sviluppatore quando serve, e non il contrario.
- [F] Che il codice dello sviluppatore invoca le funzioni del framework, che poi gli restituiscono il controllo.

Risposta: E. «Non chiamarci tu, ti chiamiamo noi»: con una libreria è il nostro codice a chiamare le funzioni, che restituiscono il controllo; con un framework il nostro codice viene innestato nel framework, che riceve la richiesta e lo invoca (per esempio i gestori delle rotte). *(Dispensa: cap. 15, «Inversione del controllo»)*

SOLUZIONI

Es. **8.** **(2 punti)** Nell'organizzazione dell'applicazione Express adottata nel corso, che cosa contiene la cartella `controllers`?

- **A** Il codice che implementa la logica di business per gestire le richieste.
- B I template Pug delle pagine.
- C I file statici: CSS, immagini, script.
- D I modelli dei dati e la connessione al database.
- E I middleware personalizzati, come quello di autenticazione.
- F La definizione dei router, che indirizzano le richieste.

Risposta: A. La struttura è: `views` (template), `routes` (router che indirizzano le richieste verso il controller giusto), `controllers` (logica di business), `models` (modelli e database), `middleware` (middleware personalizzati), `public` (file statici), e `index.js` come punto d'ingresso. Express è unopinionated: è una scelta, non una regola. *(Dispensa: cap. 16, «Organizzare un'applicazione Express»)*

Es. **9.** **(5 punti)** Si descriva la struttura di una **richiesta** e di una **risposta** HTTP, indicando le parti che le compongono e facendo un esempio per ciascuna.

Traccia della risposta.

- Richiesta: riga di richiesta (metodo, percorso, versione), intestazioni (`Host`, `User-Agent`, `Accept`, ...), riga vuota, corpo facoltativo (es. dati di un form inviato con `POST`).
- Risposta: riga di stato (versione, codice, frase), intestazioni (`Content-Type`, `Content-Length`, ...), riga vuota, corpo (es. il documento HTML).
- Esempio: `GET /index.html HTTP/1.1` con `Host: www.example.org`; risposta `HTTP/1.1 200 OK` con `Content-Type: text/html`.
- I codici di stato si raggruppano in classi (2xx successo, 3xx reindirizzamento, 4xx errore del client, 5xx errore del server).

(Dispensa: cap. 1, «I messaggi HTTP»)

Es. **10.** **(5 punti)** Si confrontino le dichiarazioni `var`, `let` e `const` in JavaScript rispetto a visibilità (scope), hoisting e possibilità di riassegnazione, con un esempio.

Traccia della risposta.

- `var`: visibilità di funzione; sollevata e inizializzata a `undefined` (leggerla prima dà `undefined`); ridichiarabile.
- `let`: visibilità di blocco; sollevata ma non inizializzata fino alla dichiarazione (TDZ, `ReferenceError`); riassegnabile.
- `const`: come `let`, ma va inizializzata e non si può riassegnare (`TypeError`); l'oggetto a cui punta resta modificabile.
- Esempio: `var` dentro un `if` è visibile fuori, `let` no.

(Dispensa: cap. 5, «Variabili»)

Es. **11.** **(5 punti)** Si descriva l'architettura basata su NGINX e PHP-FPM, spiegando in che cosa migliora l'approccio CGI.

Traccia della risposta.

- NGINX riceve tutte le richieste HTTP e serve da sé i file statici.
- Le richieste per i file `.php` vengono inoltrate, via TCP (porta 9000), a PHP-FPM.
- PHP-FPM mantiene un pool di processi worker già avviati, riusati fra le richieste; se sono tutti occupati le richieste attendono in coda.
- Rispetto a CGI non si crea un processo per richiesta; lo scripting lato server pre-elabora la richiesta (`$_GET`, `$_POST`, `$_COOKIE`) e mescola codice e HTML.

(Dispensa: cap. 10, «Un'architettura moderna: NGINX e PHP-FPM»)

Griglia delle risposte

Es.	Risposta	Capitolo	Argomento
1	C	6	array
2	D	7	ordine di esecuzione degli script
3	E	8	memorizzare dati nel browser
4	D, F	9	DNS
5	D	12	routing
6	B, C	13	sessioni web
7	E	15	framework
8	A	16	organizzazione dell'app
9	aperta	1	messaggi HTTP
10	aperta	5	variabili, scope e hoisting
11	aperta	10	scripting lato server

Punteggio massimo: 16 (chiuse) + 15 (aperte) = **31 punti**. Nelle domande con più risposte corrette ogni risposta giusta indicata vale una frazione dei punti e ogni risposta sbagliata indicata ne toglie altrettanti; il punteggio di una domanda non scende mai sotto zero.

SOLUZIONI